

Hybrid AI Development Orchestration System

Product Requirements Document (PRD)

Prepared for: Personal AI Software Engineering Platform

July 2026 · Version 1.0

[1. Executive Summary](#)

[2. Goals and Objectives](#)

[2.1 Primary Objectives](#)

[2.2 Non-Goals \(Out of Scope for v1\)](#)

[2.3 Success Criteria](#)

[3. System Architecture](#)

[3.1 High-Level Flow](#)

[3.2 Component Responsibilities](#)

[3.3 Why the Orchestrator Must Sit Between Roo Code and the Models](#)

[4. API Integration with Roo Code / Continue](#)

[4.1 Connection Model](#)

[4.2 Required Endpoint Surface](#)

[4.3 Request Handling Contract](#)

[4.4 Streaming and Long-Running Work](#)

[4.5 Tool/Function-Calling Normalization](#)

[4.6 Provider Interchangeability](#)

[5. Automatic Routing](#)

[5.1 Routing Principle](#)

[5.2 Routing Table \(Examples\)](#)

[5.3 Routing Modes](#)

[5.4 Router Decision Inputs](#)

[5.5 Model Conversation Protocol](#)

[6. Knowledge System \(Project Source of Truth\)](#)

[6.1 Purpose](#)

[6.2 Required Contents](#)

[6.3 Storage & Format](#)

[7. Live Project Memory](#)

[7.1 Behavior](#)

[7.2 Knowledge Comparison \(Proactive Regression Detection\)](#)

[8. Browser Review](#)

[8.1 Purpose](#)

[8.2 Capabilities](#)

[8.3 Workflow](#)

[8.4 Recommended Tooling](#)

[9. Vision Pipeline](#)

[9.1 Trigger](#)

[9.2 Extraction Targets](#)

[9.3 Output](#)

[10. Planner and Task Queue](#)

[10.1 Planner](#)

[10.2 Task Queue States](#)

[10.3 Resume Behavior](#)

[11. Review Loop](#)

[11.1 Automatic Trigger](#)

[11.2 Flow](#)

[11.3 Review Dimensions](#)

[12. Git Integration, Quality Gates, and Guardrails](#)

[12.1 Git Integration \(Automatic\)](#)

[12.2 Quality Gates](#)

[12.3 Guardrails \(Required for Production Readiness\)](#)

[13. Learning Layer](#)

[13.1 Behavior](#)

[13.2 Promotion Criteria](#)

[14. Long-Term Vision](#)

[15. Main Design Principle](#)

[16. Phased Implementation Plan](#)

[Phase 0 — Foundations \(Week 1-2\)](#)

[Phase 1 — Routing and Multi-Provider Support \(Week 3-4\)](#)

[Phase 2 — Knowledge Base and Memory \(Week 5-6\)](#)

[Phase 3 — Planner and Model Conversation Protocol \(Week 7-8\)](#)

[Phase 4 — Review Loop and Quality Gates \(Week 9-10\)](#)

[Phase 5 — Regression Detection \(Week 11\)](#)

[Phase 6 — Browser Review and Vision Pipeline \(Week 12-13\)](#)

[Phase 7 — Learning Layer and Continue Support \(Week 14\)](#)

[Phase 8 — Hardening \(Week 15-16\)](#)

[17. Consolidated To-Do List](#)

[Setup](#)

[API Layer](#)

[Model Adapters](#)

[Router](#)

[Knowledge Base](#)

[Memory](#)

[Planner & Task Queue](#)

[Model Conversation Protocol](#)

[Review & Quality](#)

[Git Integration](#)

[Regression Detection](#)

[Browser Review](#)

[Vision Pipeline](#)

[Learning Layer](#)

[Hardening / Guardrails](#)

[18. Open Risks and Considerations](#)

[19. Glossary](#)

1. Executive Summary

This document defines the requirements, architecture, rules, and phased implementation plan for a **personal AI software engineering platform** that unifies cloud-based reasoning models and local execution models behind a single orchestration layer.

The end user interacts with **exactly one chat interface inside VS Code** (via Roo Code, with Continue as a secondary/comparison option). Every other capability — model selection, planning, task decomposition, memory management, regression detection, browser-based QA, code review, and quality gating — happens automatically behind an **Orchestrator** service that both AI agents and the editor are unaware of as a “multi-model system.” From the editor’s point of view, it is talking to a single, very capable model.

The Orchestrator is the brain of the system. Cloud models provide intelligence (architecture, planning, review, vision). Local models provide execution (code generation, refactors, boilerplate). A persistent knowledge base and live project memory provide continuity and consistency across sessions.

2. Goals and Objectives

2.1 Primary Objectives

The system shall:

1. Operate entirely inside VS Code, through one chat interface.
2. Automatically decide which model (cloud or local) performs each task, with manual override available.
3. Maintain persistent memory of the entire project across sessions.
4. Continuously improve and review code quality without manual initiation.
5. Detect regressions automatically, without waiting for user reports.
6. Resume any project exactly where it was left off, on command.

2.2 Non-Goals (Out of Scope for v1)

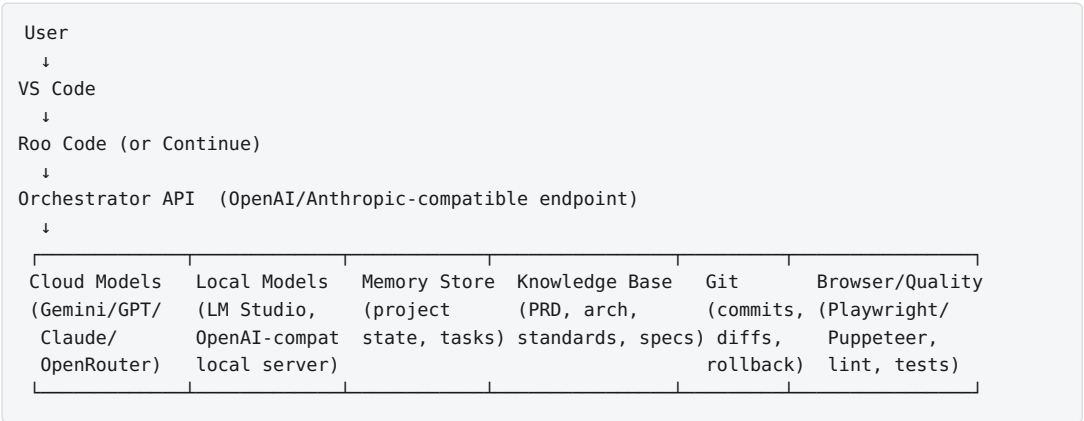
- Multi-user / team collaboration features.
- Hosting the Orchestrator as a public multi-tenant SaaS product.
- Replacing VS Code or building a custom editor.
- Supporting IDEs other than VS Code (JetBrains, etc.) in the initial release.

2.3 Success Criteria

- User can issue a single natural-language instruction and have it correctly routed to cloud or local models with no manual switching, in at least 90% of cases without override.
- Regressions introduced by local model edits are caught by the Orchestrator’s state-comparison or browser-review pipeline before the user notices, in the majority of test cases.
- A project can be closed and resumed with a single “continue project” command, with full context restored.
- Quality gates (build, lint, tests, review, regression check) block task completion until all pass.

3. System Architecture

3.1 High-Level Flow



The Orchestrator is the **only** component that talks to models directly. Cloud and local models never communicate with each other directly — all coordination is mediated by the Orchestrator, which prevents infinite loops and keeps a single point of control.

3.2 Component Responsibilities

Component	Responsibility
VS Code + Roo Code / Continue	User-facing chat interface. Sends requests to a configured “custom model provider” (the Orchestrator). Renders responses, diffs, and file edits as normal.
Orchestrator API Layer	Exposes an OpenAI-compatible (and/or Anthropic-compatible) /v1/chat/completions -style endpoint that Roo Code/Continue call as if it were a single LLM provider.
Router	Classifies incoming requests and decides Cloud / Local / Hybrid execution path.
Planner	Breaks large goals into ordered, dependency-aware tasks.
Task Queue / State Store	Tracks Completed / In Progress / Blocked / Next for every task in the project.
Memory Manager	Reads and writes live project memory before/after every execution step.

Knowledge Base	Persistent, structured source of truth (PRD, architecture, standards, decisions).
Model Adapters	Normalize requests/responses across Gemini, GPT, Claude, OpenRouter, and LM Studio's local OpenAI-compatible server.
Review Engine	Runs cloud-driven review loops against local-model output.
Regression Detector	Compares expected vs. actual project state after every meaningful change.
Browser Agent	Automates launching, navigating, and inspecting the running application for visual/functional QA.
Vision Pipeline	Routes uploaded screenshots, Figma exports, PDFs, and videos to a vision-capable cloud model for extraction into implementation tasks.
Git Manager	Automatic checkpoints, commits, diffs, and rollback.
Quality Gate	Final pass/fail authority before a task is marked complete.
Learning Layer	Detects repeated corrections and promotes them into permanent project rules.

3.3 Why the Orchestrator Must Sit Between Roo Code and the Models

Roo Code and Continue are not designed to run multi-model workflows internally — they are designed to talk to **one configured model provider** per request. To achieve “one interface, invisible orchestration,” the Orchestrator must impersonate a single model provider from the editor’s perspective while doing arbitrarily complex multi-model work internally. This is the single most important architectural constraint in this system and drives the API design in Section 4.

4. API Integration with Roo Code / Continue

4.1 Connection Model

Both Roo Code and Continue support configuring a **custom OpenAI-compatible API endpoint** as a model provider (Continue also supports an Anthropic-compatible schema). The Orchestrator will be implemented as a local (or self-hosted) HTTP server exposing this contract, so that:

- In Roo Code / Continue settings, the user points the “API Base URL” at the Orchestrator (e.g. `http://localhost:8787/v1`), instead of directly at `api.openai.com`, `api.anthropic.com`, or OpenRouter.
- The editor sends a normal chat completion request: system prompt, conversation messages, and any tool/function definitions Roo Code manages (file edit tools, terminal tools, etc.).
- The Orchestrator receives this request, treats it as the **user intent trigger**, and internally performs the full routing → planning → execution → review → memory-update cycle.
- The Orchestrator returns a single, final response in the exact OpenAI-compatible chat completion shape Roo Code expects — including correctly formatted tool calls, so Roo Code can apply file edits exactly as it would with any normal provider.

4.2 Required Endpoint Surface

Endpoint	Purpose
POST /v1/chat/completions	Primary entry point used by Roo Code / Continue. Accepts standard OpenAI chat payload; returns standard OpenAI-shaped response (or SSE stream).
GET /v1/models	Returns a single virtual model entry (e.g. orchestrator-v1) so the editor's model picker works normally.
GET /v1/health	Liveness/readiness check for local monitoring.
POST /internal/tasks/continue	Optional explicit "resume project" trigger, mappable to a special user message like "continue project" .
GET /internal/state	Debug/inspection endpoint for the current task queue and memory snapshot (not called by the editor).

4.3 Request Handling Contract

1. Parse the incoming OpenAI-style request (messages, tools, attachments).
2. Detect attachments (images, PDFs) and route them through the Vision Pipeline if present.
3. Retrieve current project memory + relevant knowledge base sections.
4. Pass the enriched context to the Router (Section 5).
5. Execute the resulting plan (cloud/local/hybrid), including any review loops.
6. Update memory and knowledge base.
7. Translate the final result into a valid OpenAI-compatible response, including tool calls formatted the way Roo Code's file-edit and terminal tools expect.

4.4 Streaming and Long-Running Work

Multi-step orchestration (architecture → implementation → review → fix → re-review) can take minutes, while Roo Code expects a responsive chat stream. Requirements:

- The Orchestrator **must** support Server-Sent Events (SSE) streaming responses, even if the underlying multi-step process is not itself token-by-token.
- While internal steps are running, the Orchestrator streams lightweight **status tokens** (e.g. "Planning...", "Implementing auth module...", "Reviewing...") so the VS Code UI does not appear frozen, then streams the final structured result once ready.
- Long jobs (> ~60s) should also be tracked in /internal/state so a dropped connection does not lose progress; the user can ask "continue" to resume.

4.5 Tool/Function-Calling Normalization

Roo Code and Continue each have slightly different tool-calling schemas for file edits, terminal commands, and diffs. The Orchestrator's Model Adapter layer must:

- Normalize outbound requests to each cloud/local provider's own tool-calling format.
- Normalize the final result back into whichever tool-calling schema the *calling* agent (Roo Code vs. Continue) expects, since the two are not identical.

4.6 Provider Interchangeability

Cloud provider adapters (Gemini, GPT, Claude, OpenRouter) and the local adapter (LM Studio’s OpenAI-compatible server) must be swappable via configuration, not code changes. Each adapter implements a common internal interface:

```
ModelAdapter {
  complete(request: NormalizedRequest): NormalizedResponse
  supportsVision: boolean
  supportsTools: boolean
  contextWindow: number
}
```

5. Automatic Routing

5.1 Routing Principle

Cloud models handle **intelligence-heavy, low-volume-output** tasks. Local models handle **execution-heavy, high-volume-output** tasks. The user never manually selects a model in normal operation.

5.2 Routing Table (Examples)

User Intent	Route
Design authentication system	Cloud
Generate 40 React components	Local
Review UI	Cloud
Fix issues / apply suggested fixes	Local
Analyze this PDF / screenshot / video	Cloud (Vision Pipeline)
Generate code from an approved plan	Local
Security / performance / architecture review	Cloud
Boilerplate, formatting, documentation generation	Local
Regression / diff analysis	Cloud

5.3 Routing Modes

- **Auto** (default): Router decides automatically per request.
- **Cloud Only**: Forces all reasoning and generation through the cloud model (e.g. for small, high-stakes changes).
- **Local Only**: Forces all generation through the local model (e.g. offline mode, cost control).

5.4 Router Decision Inputs

The Router should classify each request using:

- Task verbs (design, review, analyze → cloud; generate, fix, refactor, format → local).
- Estimated output size (large multi-file output → local; short structured output → cloud).

- Presence of attachments (images/PDF/video → cloud vision).
- Explicit user override keywords (e.g. “use cloud”, “use local”).
- Current mode setting (Auto / Cloud Only / Local Only).

5.5 Model Conversation Protocol

Cloud and local models never talk directly. All coordination is mediated:

```
Cloud → Orchestrator → Local → Orchestrator → Cloud
```

Standard workflow:

1. Cloud creates architecture / plan.
2. Local implements it.
3. Cloud reviews implementation.
4. Local fixes flagged problems.
5. Cloud validates again.
6. Repeat until the Quality Gate (Section 10) passes, or a maximum iteration cap is hit (see Section 12.3, Guardrails).

6. Knowledge System (Project Source of Truth)

6.1 Purpose

LLMs must never rely on conversation history alone. A persistent, structured Knowledge Base is the project’s single source of truth, retrieved and injected into every model call.

6.2 Required Contents

- Product Requirements Document (PRD)
- Architecture documentation
- UI guidelines / design system
- Folder structure conventions
- Coding standards
- API specifications
- Business rules
- Decision log
- Completed work log
- Pending work log

6.3 Storage & Format

- Stored as versioned structured documents (Markdown + JSON metadata) inside the project repository, so it travels with Git.
- Indexed for retrieval (embedding-based or structured lookup) so only relevant sections are injected per task, avoiding context-window bloat.
- Every write to the Knowledge Base is itself a Git-tracked change.

7. Live Project Memory

7.1 Behavior

Memory is continuous, not session-scoped.

Before every execution, the Orchestrator retrieves:

- Project state
- Completed tasks
- Active tasks
- Pending tasks
- Important files
- Previous decisions
- Recent changes

This context is injected into both cloud and local model calls.

After every execution, the Orchestrator updates memory — after every meaningful step, not only at session end. Models always operate against the newest project state.

7.2 Knowledge Comparison (Proactive Regression Detection)

Memory must do more than store history — it must **compare expected project state against current project state** after every significant change.

Example: if yesterday's known-good state included Login, Dashboard, and Navbar, and today's change was "fix the Login button," the Orchestrator should automatically detect if:

- Navbar disappeared
- Dashboard broke
- Authentication routing changed
- Any existing functionality regressed

This comparison happens automatically — the user should never have to report these problems manually. This includes static comparison (file/route/component diffing against the expected state) and, where static comparison is insufficient, triggering the Browser Review pipeline (Section 8).

8. Browser Review

8.1 Purpose

Some regressions are only observable at runtime and cannot be detected by comparing source-level project knowledge alone.

8.2 Capabilities

The Browser Agent shall be able to:

- Launch the application
- Navigate between pages
- Click interactive elements

- Capture screenshots
- Inspect the browser console for errors
- Inspect network requests/responses
- Inspect layout
- Inspect responsiveness across viewport sizes

8.3 Workflow

1. Browser Agent gathers evidence (screenshots, console logs, network traces).
2. The evidence is sent to the cloud model for analysis.
3. If issues are found, the cloud model creates fix tasks.
4. Local model applies fixes.
5. Browser Agent re-tests.
6. Loop continues until clean or the iteration cap is reached.

8.4 Recommended Tooling

Playwright (preferred) or Puppeteer, run headless in CI-like fashion, orchestrated as an internal Orchestrator tool rather than exposed directly to the user.

9. Vision Pipeline

9.1 Trigger

Any uploaded screenshot, Figma design export, PDF, video, or other UI reference automatically routes to the vision-capable cloud model — no manual instruction required.

9.2 Extraction Targets

- Layout
- Colors
- Spacing
- Interactions
- Requirements
- UI structure

9.3 Output

The cloud model's extraction is converted directly into implementation tasks in the Task Queue, which the local model then implements, following the standard Model Conversation Protocol (Section 5.5).

10. Planner and Task Queue

10.1 Planner

Large goals automatically decompose into smaller, dependency-ordered tasks.

Example — “Build an e-commerce app”:

Authentication → Home → Products → Cart → Orders → Payments → Admin → Testing → Deployment

The Planner is responsible for managing dependencies between tasks (e.g. Cart depends on Products and Authentication).

10.2 Task Queue States

Every task exists in exactly one state at a time:

- **Completed**
- **In Progress**
- **Blocked** (with recorded blocking reason/dependency)
- **Next**

10.3 Resume Behavior

If development stops at any point, a single user command — **“continue project”** — must resume execution exactly where it left off, using the Task Queue and Live Project Memory, with no re-explanation required from the user.

11. Review Loop

11.1 Automatic Trigger

Every completed task automatically enters review. The user never manually initiates review.

11.2 Flow

```
Cloud: Review → Create improvement list
↓
Local: Apply fixes
↓
Cloud: Approve (or repeat)
```

11.3 Review Dimensions

- Correctness against the task spec
- UI/UX quality
- Design system / style-guide adherence
- Security
- Performance
- Architectural consistency with the Knowledge Base

12. Git Integration, Quality Gates, and Guardrails

12.1 Git Integration (Automatic)

- Checkpoint before major edits.
- Create meaningful, descriptive commits per completed task.
- Roll back failed implementations automatically if the Quality Gate fails after the maximum retry count.
- Compare versions / inspect diffs as part of the Review Loop and Regression Detector.

12.2 Quality Gates

A task is not marked complete until **all** of the following pass:

- Build succeeds
- Tests pass
- Lint passes
- Cloud review passes
- Architecture remains consistent with the Knowledge Base
- No regressions detected (state comparison + browser review where applicable)

12.3 Guardrails (Required for Production Readiness)

These are not explicitly listed in the original vision document but are necessary implementation requirements:

- **Iteration caps** on Cloud ↔ Local review loops (e.g. max 5 rounds) to prevent infinite fix/review cycles, with escalation to the user if the cap is hit.
- **Cost/token budgets** per task and per session, configurable per provider, to prevent runaway cloud API spend.
- **Sandboxed execution** for any local terminal/build commands the local model or Browser Agent triggers.
- **Diff-size limits** before requiring explicit user confirmation on very large automated changes.

13. Learning Layer

13.1 Behavior

If the cloud model repeatedly issues the same correction across tasks, the Orchestrator promotes that correction into a **permanent project rule**, stored in the Knowledge Base.

Examples:

- “Always use React Query.”
- “Never use inline CSS.”
- “Always use Tailwind utility classes.”
- “Always follow Atomic Design.”

13.2 Promotion Criteria

A correction should be considered for promotion after appearing a defined threshold number of times (e.g. 3 occurrences) across distinct tasks, and should be surfaced to the user for confirmation before being written as a hard rule, to avoid over-fitting to one-off situations.

14. Long-Term Vision

The Orchestrator should evolve into a complete AI software engineering platform, capable of acting as:

- Project Manager
- Tech Lead / Senior Software Architect
- Frontend Engineer
- Backend Engineer
- QA Engineer
- UI Reviewer
- DevOps Assistant
- Documentation Writer

Each responsibility is delegated to the most suitable model, while the user continues to experience one seamless conversation.

15. Main Design Principle

The user interacts with only one AI interface. Everything else — routing, planning, memory, reviews, browser testing, quality control, and inter-model communication — happens automatically behind the scenes through the Orchestrator.

- The Orchestrator is the brain.
- Cloud models provide intelligence.
- Local models provide execution.
- Memory provides continuity.
- Knowledge provides consistency.
- Browser tools provide validation.

Together, they function as one autonomous AI software engineering team.

16. Phased Implementation Plan

Phase 0 — Foundations (Week 1-2)

- Stand up the Orchestrator as a local HTTP server.
- Implement `/v1/chat/completions` and `/v1/models` with a stubbed single-model passthrough (no routing yet).
- Connect Roo Code to the Orchestrator as a custom provider; validate basic chat + file-edit tool calls round-trip correctly.
- Implement LM Studio adapter (local) and one cloud adapter (e.g. Claude) behind the common `ModelAdapter` interface.

Exit criteria: Roo Code can send a message and get a working response through the Orchestrator, from either the local or one cloud model, manually selected.

Phase 1 — Routing and Multi-Provider Support (Week 3-4)

- Implement the Router with the classification rules in Section 5.

- Add remaining cloud adapters (Gemini, GPT, OpenRouter).
- Implement Auto / Cloud Only / Local Only modes.
- Implement basic SSE streaming with status tokens.

Exit criteria: A representative set of test prompts route correctly without manual model selection.

Phase 2 — Knowledge Base and Memory (Week 5-6)

- Implement structured Knowledge Base storage (Markdown + metadata) in-repo.
- Implement retrieval (embedding or structured lookup) and context injection.
- Implement Live Project Memory read/write around every execution step.
- Implement Task Queue with Completed / In Progress / Blocked / Next states.

Exit criteria: “Continue project” resumes correctly after a full session restart.

Phase 3 — Planner and Model Conversation Protocol (Week 7-8)

- Implement the Planner for large-goal decomposition with dependency tracking.
- Implement the Cloud → Orchestrator → Local → Orchestrator → Cloud conversation protocol.
- Implement iteration caps and escalation guardrails (Section 12.3).

Exit criteria: A multi-step goal (e.g. “build authentication”) is decomposed, implemented, and iterated on without manual intervention, and stops safely when capped.

Phase 4 — Review Loop and Quality Gates (Week 9-10)

- Implement automatic Review Loop triggering on task completion.
- Implement Quality Gate checks: build, test, lint, review, architecture consistency.
- Implement Git auto-checkpoint, auto-commit, and rollback-on-failure.

Exit criteria: No task can be marked complete without passing all gates; failed tasks roll back cleanly.

Phase 5 — Regression Detection (Week 11)

- Implement Knowledge Comparison (expected vs. current state) after every significant change.
- Surface detected regressions as automatic fix tasks fed back into the Model Conversation Protocol.

Exit criteria: A deliberately introduced regression (e.g. breaking the Navbar while fixing Login) is caught automatically.

Phase 6 — Browser Review and Vision Pipeline (Week 12-13)

- Integrate Playwright as the Browser Agent.
- Implement screenshot/console/network/responsiveness capture and cloud analysis.
- Implement the Vision Pipeline for screenshots, Figma exports, PDFs, and video, converting extraction output into tasks.

Exit criteria: Uploading a design reference produces correctly scoped implementation tasks; a UI regression is caught via browser evidence.

Phase 7 — Learning Layer and Continue Support (Week 14)

- Implement repeated-correction detection and rule-promotion workflow (with user confirmation).
- Add Continue as a secondary agent integration, validating tool-call schema normalization (Section 4.5).

Exit criteria: A repeated correction becomes a permanent project rule; Continue can drive the same Orchestrator with equivalent behavior to Roo Code.

Phase 8 — Hardening (Week 15-16)

- Cost/token budget enforcement.
- Sandboxed execution for local commands.
- Diff-size confirmation thresholds.
- Load/latency testing of the streaming endpoint under realistic multi-step orchestration times.

Exit criteria: System operates reliably under normal daily use without manual babysitting.

17. Consolidated To-Do List

Setup

- ☐ Install and configure LM Studio; expose local OpenAI-compatible endpoint.
- ☐ Choose initial cloud provider(s) and obtain API keys (Gemini / GPT / Claude / OpenRouter).
- ☐ Install Roo Code in VS Code; install Continue as secondary comparison.
- ☐ Scaffold Orchestrator service (language/runtime of choice) with health check endpoint.

API Layer

- ☐ Implement `/v1/chat/completions` (non-streaming) with stub routing.
- ☐ Implement `/v1/models`.
- ☐ Point Roo Code's custom provider setting at the Orchestrator; validate round-trip.
- ☐ Implement SSE streaming with status-token support.
- ☐ Implement `/internal/state` and `/internal/tasks/continue`.
- ☐ Normalize tool-calling schema differences between Roo Code and Continue.

Model Adapters

- ☐ Define common `ModelAdapter` interface.
- ☐ Implement LM Studio adapter (local).
- ☐ Implement Claude adapter.
- ☐ Implement GPT adapter.
- ☐ Implement Gemini adapter.
- ☐ Implement OpenRouter adapter.
- ☐ Add vision support flags per adapter.

Router

- ☐ Implement intent classification rules (Section 5.2).

- ☐ Implement Auto / Cloud Only / Local Only modes.
- ☐ Implement override keyword detection.

Knowledge Base

- ☐ Define Markdown + metadata schema for PRD, architecture, standards, decisions, etc.
- ☐ Implement retrieval/indexing for relevant-section injection.
- ☐ Version Knowledge Base changes through Git.

Memory

- ☐ Implement pre-execution context retrieval.
- ☐ Implement post-execution memory update (every meaningful step).
- ☐ Implement Knowledge Comparison for regression detection.

Planner & Task Queue

- ☐ Implement goal decomposition into dependency-ordered tasks.
- ☐ Implement Completed / In Progress / Blocked / Next state tracking.
- ☐ Implement “continue project” resume flow.

Model Conversation Protocol

- ☐ Implement Cloud → Orchestrator → Local → Orchestrator → Cloud message passing.
- ☐ Implement iteration caps and escalation-to-user on cap hit.

Review & Quality

- ☐ Implement automatic Review Loop on task completion.
- ☐ Implement build / test / lint hooks.
- ☐ Implement architecture-consistency check against Knowledge Base.
- ☐ Implement Quality Gate as the single completion authority.

Git Integration

- ☐ Auto-checkpoint before major edits.
- ☐ Auto-commit on task completion with meaningful messages.
- ☐ Auto-rollback on Quality Gate failure after max retries.
- ☐ Diff/version comparison utilities.

Regression Detection

- ☐ Static state comparison (expected vs. current) after every significant change.
- ☐ Auto-generate fix tasks for detected regressions.

Browser Review

- ☐ Integrate Playwright as the Browser Agent.
- ☐ Implement screenshot, console, network, and responsiveness capture.
- ☐ Feed evidence to cloud model for analysis and fix-task generation.

Vision Pipeline

- ☐ Implement attachment detection (image/PDF/video) on incoming requests.
- ☐ Route attachments to vision-capable cloud model.
- ☐ Convert extraction output into Task Queue entries.

Learning Layer

- ☐ Track repeated corrections across tasks.
- ☐ Implement promotion threshold and user-confirmation step.
- ☐ Write promoted rules into the Knowledge Base as permanent standards.

Hardening / Guardrails

- ☐ Cost/token budget enforcement per task/session.
- ☐ Sandboxed execution for terminal/build commands.
- ☐ Diff-size confirmation threshold for large automated changes.
- ☐ Latency/load testing of streaming under long multi-step jobs.

18. Open Risks and Considerations

Risk	Mitigation
Roo Code / Continue tool-calling schemas drift or differ enough to break normalization	Maintain adapter-level integration tests against both agents on each release.
Long orchestration cycles cause editor-side timeouts	SSE status streaming, task-state persistence, and explicit “continue” resume.
Infinite or excessive Cloud ↔ Local review loops	Hard iteration caps with user escalation.
Cloud API cost overruns from frequent review cycles	Token/cost budgets per task and session, configurable alerts.
False-positive regression detection blocking valid work	Allow user override/approval at the Quality Gate when a flagged “regression” is confirmed intentional.
Local model quality insufficient for complex logic despite cloud planning	Router escalates repeated local failures on a task back to cloud-assisted implementation.

19. Glossary

- **Orchestrator** — The central service mediating all model calls, memory, knowledge, review, and quality control.
- **Router** — Sub-component of the Orchestrator that decides Cloud vs. Local execution per task.
- **Knowledge Base** — Persistent, versioned source of truth for the project (PRD, architecture, standards, decisions).
- **Live Project Memory** — Continuously updated state of tasks, files, and recent changes, injected into every model call.
- **Quality Gate** — The final automated pass/fail check before a task can be marked complete.

- **Model Conversation Protocol** — The mediated Cloud ↔ Orchestrator ↔ Local message-passing pattern used for plan/implement/review cycles.